

1. **Question:** Demonstrate how a child class can access a protected member of its parent class within the same package. Explain with example what happens when the child class is in a different package.

Answer:

In java, a protected member can be accessed:

- Directly in subclasses within the same package
- In subclasses of different packages, but only through inheritance.

Case 1: Same Package Access

```
// Parent.java (my package)
```

```
package mypackage;
```

```
public class Parent {
```

```
    protected String msg = "Hello from Parent!";
```

```
}
```

```
// child.java (my package)
```

```
package mypackage;
```

```
public class Child extends Parent {
```

```
    public void show() {
```

```
        System.out.println(msg); // accessible directly
```

```
    }
```

Output: Hello from Parent !

Case 2: Different Package Access

//Parent.java (mypackage)

package mypackage;

public class Parent {

protected String msg = "Hello from Parent !";

}

//child.java (anotherpackage)

package anotherpackage;

import mypackage . Parent;

public class child extends Parent {

public void show () {

System.out.println (msg); // accessible via inheritance.

}

}

Output: Hello from Parent !

But the following will cause error:

Parent p = new Parent ();

System.out.println (p.msg); // Not allowed outside package

Conclusion:

protected members are accessible within the same package and in subclasses. In a different

package, access is allowed only via inheritance, not via object reference.

2. Question: Compare abstract classes and interfaces in terms of multiple inheritance. When would you prefer to use an abstract class and when an interface?

Answer: In java, multiple inheritance of implementation is not allowed using classes but is possible using interfaces.

Feature	Abstract class	Interface
Inheritance Type	Single inheritance	Multiple inheritance supported
Method Types	Abstract & concrete	Abstract (default / static in Java 8+)
Constructors	Yes	No
Variables	Can have instance vars	Only public static final vars
When to use	Common base functionality	Capability extension / mix-ins

Example of multiple Inheritance using Interface

```
Interface A {
    void show ();
}
```

```
interface B {
    void display ();
}
```

```
class C implements A, B {
    public void show () { System.out.println("\nFrom A"); }
    public void display () { System.out.println("\nFrom B"); }
}
```

Output: From A
From B

When to Use what :

Use Abstract Class When:

- You want to share common code / fields .
- You expect future subclasses to inherit and extend behavior .

Use Interface when:

- You need multiple inheritance .
- You define capabilities (e.g. Runnable, Comparable)

- Your class may implement multiple unrelated types.

Conclusion: Interfaces allow multiple inheritance, making them ideal for defining capabilities. Abstract classes are best for shared logic and providing a partial base implementation.

3. Question: How does encapsulation ensure data security and integrity? Show with a BankAccount class using private variables and validated methods such as `setAccountNumber(String)` etc.

Answer: Encapsulation is the concept of wrapping data (variables) and code (methods) together as a single unit. In java, this is achieved using private variable and public getter/setter methods. It ensures:

- Data hiding from direct external access.
- Validation before updating data, maintaining integrity.

Example : BankAccount.java

```
public class BankAccount {  
    private String accountNumber;  
    private double balance;  
  
    public void setAccountNumber (String accNum) {  
        if (accNum != null && !accNum.trim().isEmpty()) {  
            accountNumber = accNum;  
        } else {  
            System.out.println("Invalid account number.");  
        }  
    }  
  
    public void setInitialBalance (double amount) {  
        if (amount >= 0) {  
            balance = amount;  
        } else {  
            System.out.println("Balance cannot be  
                                negative.");  
        }  
    }  
}
```


4. Question: Write Program to -

- i) Find the kth smallest element in an ArrayList.
- ii) Create a TreeMap to store the mappings of words to their frequencies in a given text.
- v) Check if two LinkedLists are equal

Answers: (i)

```
import java.util.*;

public class KthSmallest {
    public static int findKth (ArrayList <Integer> list,
                               int k) {
        Collections.sort (list);
        return list.get (k-1);
    }

    public static void main (String [] args) {
        ArrayList <Integer> list = new ArrayList <> (Arrays.
            asList (7, 2, 4, 9, 1));
        System.out.println ("3rd Smallest : " + findKth (list, 3));
    }
}
```

(ii) TreeMap for Word Frequencies

```

import java.util.*;

public class WordFrequency {
    public static void main (String [] args) {
        String text = "java is easy and java is powerful";
        String [] words = text.split(" ");

        TreeMap < String , Integer > freq = new TreeMap <>();

        for (String word : words) {
            freq.put (word, freq.getOrDefault (word, 0) + 1);
        }

        System.out.println ("Word Frequencies: " + freq);
    }
}

```


v) Check if Two LinkedLists Are Equal

```
import java.util.*;

public class CompareLinkedLists {

    public static void main (String [] args) {

        LinkedList <Integer> list1 = new LinkedList <> (
            Arrays.asList (1, 2, 3));

        LinkedList <Integer> list2 = new LinkedList <> (
            Arrays.asList (1, 2, 3));

        if (list1.equals (list2)) {
            System.out.println ("Both LinkedLists are equal.")
        } else {
            System.out.println ("LinkedLists are not equal.");
        }
    }
}
```

5. Question: Developing a multithreading-based project to simulate a car parking management system.

Answer:

Code Implementation:

// RegistrarParking.java

```
public class RegistrarParking {
    private final String carNumber;

    public RegistrarParking(String carNumber) {
        this.carNumber = carNumber;
    }

    public String getCarNumber() {
        return carNumber;
    }
}
```

// ParkingPool.java

```
import java.util.LinkedList;

public class ParkingPool {
    private final LinkedList<RegistrarParking> queue =
        new LinkedList<>();

    public synchronized void requestParking(RegistrarParking
        car) {
        queue.addLast(car);
        System.out.println("Car " + car.getCarNumber() + "
            requested parking.");
        notifyAll();
    }
}
```



```

public synchronized RegistrarParking getNextCar() {
    while (queue.isEmpty()) {
        try {
            wait();
        } catch (InterruptedException ignored) {}
    }
    return queue.removeFirst();
}
}

```

// ParkingAgent.java

```

public class ParkingAgent extends Thread {
    private final int agentId;
    private final ParkingPool pool;

```

```

    public ParkingAgent(int id, ParkingPool pool) {
        this.agentId = id;
        this.pool = pool;
    }

```

```

    public void run() {
        while (true) {

```

```

            RegistrarParking car = pool.getNextCar();
            System.out.println("Agent " + agentId + " parked car" +
                + car.getCarNumber() + ".");

```

```

            try { Thread.sleep(500); // simulate parking time

```

```

    } catch (InterruptedException ignored) { }
  }
}

```

// MainClass.java

```

public class MainClass {
    public static void main (String [] args) {
        parkingPool pool = new ParkingPool ();

        // start two agents
        new ParkingAgent (1, pool).start ();
        new ParkingAgent (2, pool).start ();
        String [] cars = {"ABC123", "XYZ456", "DEF789", "GHI1000"};
        for (String car : cars) {
            new Thread () -> {
                pool.requestParking (new RegistrarParking (car));
            }.start ();
        }
    }
}

```

Sample Output :

Car ABC123 requested parking.
 Car XYZ456 requested parking.
 Agent 1 parked car ABC123
 Agent 2 parked car XYZ456

6. Question: How does Java handle XML data using DOM and SAX parsers? Compare both

Answer:

Feature	DOM	SAX
memory	High (loads whole XML)	Low (reads line by line)
speed	Slower for big files	faster for large files
Navigation	Easy (tree structure)	Hard
notification modification	Yes	No
Best for	Small XML, editing	Large XML

7. Question: How does the virtual DOM in React improve performance? Compare it with the traditional DOM and explain the diffing algorithms with a simple component update example.

Answer: Virtual DOM in React: Performance Boost
How it improves performance:

Instead of updating the real DOM directly (which is slow) React:

1. Updates the VDOM first.
2. Differs it with the previous VDOM
3. Applies minimal real DOM changes.

Feature	Traditional DOM	Virtual DOM (React)
Updates	Directly updates DOM	Differs and batches updates
Speed	Slower (repaints often)	Faster (optimized updates)
Efficiency	Low for frequent changes	High for dynamic UIs

Diffing Algorithm:

- React compares the old and new VDOM trees.
- Finds minimal changes (e.g. changed text, moved node)
- Applies only those to the real DOM (patching)

Example:

// old VDOM

<h1>Hello </h1>

// New VDOM

<h1>Hello, World </h1>

8. Question: What is event delegation in JavaScript, and how does it optimize performance?

Answer: Event Delegation is a technique where a single event listener is added to a parent element instead of multiple child elements.

Performance Benefits:

Reduces memory usage, Works with dynamically added elements, Improves performance, especially with large DOM trees.

Example: Click on Dynamic List Items

```
<ul id="myList">
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
```

```
<script>
```

```
  document.getElementById("myList").addEventListener
```

```
    ("click", function(e) {
```

```
      if (e.target.tagName === "LI") {
```

```
        alert("Clicked: " + e.target.textContent);
```

```
      }
    });
```

// Dynamically add a new

```
const newItem = document.createElement("li");
```



```

newItem.textContent = "Item 3" ;
document.getElementById ("myList").appendChild (
    newItem);
</script>

```

9. Question : Explain how java Regular Expressions can be used for input validation. Write a regex pattern to validate an email address and describe how it works using the pattern and Matcher classes.

Solution:

Java Provides the Pattern and Matcher classes in the java.util.regex package to validate inputs using regular expressions (regex)

Regex Pattern to Validate Email

String regex = "^[A-Za-z0-9+.-]+@[A-Za-z0-9.-]+;\$";

How it works :

- ^ and \$: Start and end of the String
- [A-Za-z0-9+.-]+ : Allows letters, digits, +, -, ., and - before the @
- @ : Must include an @
- [A-Za-z0-9.-]+ : Allows domain name characters after @.

10. Question: What are custom annotations in Java, and how can they be used to influence program behavior at runtime using reflection?

Answer: Custom Annotations allow developers to define their own metadata, which can influence program behavior during runtime using Java Reflection API

Define a custom Annotation:

```
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface RunImmediately {
    String value() default "Task";
}
```

Apply Annotation in a class:

```
public class Democlass {
    @RunImmediately("Initialize")
    public void start() {
        System.out.println("Starting the system...");
    }
    public void stop() {
        System.out.println("Stopping the system...");
    }
}
```

Process Annotation Using Reflection :

```
import java.lang.reflect.*;

public class AnnotationProcessor {
    public static void main(String[] args) throws Exception {
        DemoClass obj = new DemoClass();

        for (Method method : DemoClass.class.getDeclaredMethods()) {
            if (method.isAnnotationPresent(RunImmediately.class)) {
                RunImmediately ann = method.getAnnotation(
                    RunImmediately.class);
                System.out.println("Running: " + ann.value());
                method.invoke(obj);
            }
        }
    }
}
```


11. Question: Singleton Design Pattern (Java)

Answer: Purpose: Ensures only one instance of a class exists.

```
public class Singleton {
    private static Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null)
            instance = new Singleton();
        return instance;
    }
}
```

Thread-Safe:

```
public class Singleton {
    private Singleton() {}
    private static class Holder {
        static final Singleton INSTANCE = new Singleton();
    }
    public static Singleton getInstance() {
        return Holder.INSTANCE;
    }
}
```

12. Question: Describe how JDBC manages communication between a Java application and a relational database

Answer: JDBC enables Java applications to connect and interact with relational databases using SQL.

Steps to execute a SELECT Query:

1. Load JDBC Driver
2. Establish Connection
3. Create statement
4. Execute query
5. Process ResultSet
6. Close Resources

Example code:

```
import java.sql.*;
```

```
public class SelectExample {
    public static void main (String [] args) {
        Connection conn = null;
        Statement stmt = null;
        ResultSet rs = null;
```

```
try {
```

```
    Class.forName("com.mysql.cj.jdbc.Driver");
```



```
conn = DriverManager.getConnection ("jdbc : mysql : // local-  
- host : 3306 / mydb ", "user", "password");
```

```
stmt = conn.createStatement();
```

```
rs = stmt.executeQuery ("SELECT * FROM students");
```

```
while (rs.next()) {  
    System.out.println ("Name : " + rs.getString ("name"));
```

```
}  
} catch (Exception e) {
```

```
    e.printStackTrace();
```

```
} finally {
```

```
    try { if (rs != null) rs.close(); } catch (Exception e) {
```

```
    try { if (stmt != null) stmt.close(); } catch (Exception e) {
```

```
    try { if (conn != null) conn.close(); } catch (Exception e) {
```

```
    }
```

```
}
```

13. Question: How do servlets and JSPs work together in a web application following the MVC architecture?

Answer: In Java web applications, MVC separates concerns:

Model: Java classes containing business logic.

View: JSP files for user interface.

Controller: servlets handle requests, update model, and forward to JSP.

Use case: Student Info Display

1. Model - Student.java

```
public class Student {
    private String name;
    private int age;
}
```

2. Controller - StudentServlet.java

```
@WebServlet("/student")
```

```
public class StudentServlet extends HttpServlet {
    protected void doGet (HttpServletRequest req,
        HttpServletResponse resp) throws ServletException,
        IOException {
        Student s = new Student ("Nusrat", 22);
```



```

req.setAttribute("student", s);
req.getRequestDispatcher("student.jsp").forward(req, res);
}
}

```

3. View- student.jsp

```

<%@page import="model.Student"%>
<%
    student s = (Student) request.getAttribute("student");
%>
<% = s.getName() %> </P>
<P> Age: <% = s.getAge() %> </P>

```

14. Question: Explain the life cycle of a Java Servlet.
What are the roles of the `init()`, `service()`, and `destroy()` methods?

Answer: A Servlet follows this 3-phase life cycle managed by the servlet container;

1. `init()`

- Called once when the servlet is first loaded.
- Used for initialization tasks

2. service ()

- Called for each client request.
- Handles logic based on request type (doGet(), doPost(), etc).

3. destroy ()

- Called once when the servlet is being destroyed.
- Used to release resources (e.g. closing DB connections).

Handling Concurrent Requests

- Servlets use a multithreaded model.
- For each request, the container spawns a new thread, not a new servlet object.

Thread Safety Issues

- If multiple threads access shared resources without synchronization, data inconsistency may occur.

Solution:

Avoid shared mutable state or use synchronized Atomic classes or thread-local variables.

15. Question: A single instance of a servlet handles multiple requests using threads. What problems can occur if shared resources are accessed by multiple threads?

Answer: A single servlet instance handles multiple concurrent requests using separate threads. If shared (instance) variables are accessed or modified by these threads without proper control, it leads to race conditions and inconsistent behavior.

Example (Unsafe servlet)

```
public class CounterServlet extends HttpServlet {
    private int counter = 0;
    protected void doGet (HttpServletRequest req, HttpServletResponse res) throws IOException {
        counter++;
        res.getWriter().println ("Visitor count: " + counter);
    }
}
```

Problem: If two threads increment counter simultaneously, updates can be lost due to race conditions.

Solution: Use synchronized Block

```
protected void doGet (HttpServletRequest req,
    HttpServletResponse res) throws IOException {
    synchronized (this) {
        counter++;
        res.getWriter().println ("Visitor Count: "+counter);
    }
}
```

16. Question: Describe how the MVC (Model-View-Controller) pattern separates concerns in a Java web application. Explain the advantages of this structure in terms of maintainability and scalability.

Answer: MVC separates concerns as follows:

- Model : Manages data and business logic.
- View: Handles UI display.
- Controller : Processes user input and updates model / view

Student Registration Example

1. User submits form → register.jsp (View)
2. Servlet receives data → StudentServlet.java (Controller)
3. Servlet uses Student.java to store data (Model)
4. Servlet forwards success messages back to JSP

Advantages :

- Maintainability : Logic / UI separated ; changes are isolated.
- Scalability : Easy to add new features without rewriting entire code.
- Testability : Model can be tested independently of UI.

17. Questions: In a Java EE application, how does a servlet controller manage the flow between the model and the view? Provide---

Answer: MVC in Java EE

servlet = Controller : handles request

Model = Java class : holds data

JSP = View : shows result

Example :

```
// Model
public class Student {
    String name;
    public Student (String name) { this.name = name; }
    public String getName () { return name; }
}
```

// controller

@WebServlet ("/register")

```
public class RegisterServlet extends HttpServlet {
    protected void doPost (HttpServletRequest req,
        HttpServletResponse res) throws ServletException,
        IOException {
```



```

Student s = new Student (req.getParameter("name"));
req.setAttribute("student", s);
req.getRequestDispatcher("result.jsp").forward(req,
res);
}
}

```

jsp

<!-- View-->

<% = ((student) request.getAttribute("student")).getName() %>

18. Question: Compare and contrast cookies, URL rewriting and HttpSession as methods for session tracking in servlets. Discuss their advantages, limitations and ideal use cases.

Answer:

Method	Advantages	Limitations	Best Use Case
cookies	Stores data on client; simple	Can be disabled; size limit	Tracking users across visits
URL Rewriting	Works when cookies disabled	Manual link rewriting; insecure	Bookmarkable links with session data
HttpSession	Server-managed; secure & easy	Memory load on server	Most common for user login/session

19. Questions: A web application stores user login information using HttpSession. Explain.

Answer: HttpSession Workflow :

- When user logs in, server creates a HttpSession object and stores user data.
- A session ID is sent to the client via a cookie
- On next requests, this ID is sent back and server retrieves the session.

Session Timeout :

- Set by session.setMaxInactiveInterval (seconds) or in web.xml.
- If user is inactive beyond the timeout, session auto-expires.

Manual Invalidation :

- Use session.invalidate() to logout securely.
- Prevents unauthorized access
- Always clear session on logout or after sensitive actions.

20. Question : Explain how Spring MVC handles an HTTP request from a browser. Describe the role of the @Controller, @RequestMapping, and Model objects in separating business logic from presentation.....

Answer :

1. User submits login form → HTTP request sent to server

2. @Controller

- Class marked with @Controller handles web request.

@Controller

```
public class LoginController {....}
```

3. @RequestMapping

- Binds URL to a method.

4. Model

- Passes data from controller to view

```
model.addAttribute("user", user);
```

5. View Resolver

- Returns view to the browser.

21. Question: Spring MVC uses the DispatcherServlet as a front controller. Describe its role in the request processing workflow.

Answer: Role:

Acts as the central controller that receives all HTTP requests and delegates them to appropriate components.

Workflow:

1. Request Comes In:

Browser sends request → DispatcherServlet catches it.

2. Handler Mapping:

Finds the correct @Controller and method using URL (via @RequestMapping)

3. Controller Executes:

Controller processes data → returns view name and Model.

4. View Resolver:

Dispatcher uses ViewResolver to find actual JSP/HTML file based on view name.

5. Render Response:

view is rendered and sent back to the browser.

22. Question: How does PreparedStatement improve performance and security over statement in JDBC? Write a short example

Answer: PreparedStatement vs Statement (JDBC)

Security:

- PreparedStatement protects against SQL Injection by precompiling the query and safely inserting parameters.

Performance:

- SQL is precompiled and reused → faster for repeated queries.

Example: Insert Using PreparedStatement

```
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/mydb", "root", "password");
```

```
String sql = "INSERT INTO students (name, email)
    VALUES (?, ?)";
```

```
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, "Alice");
```



```
ps.setString(2, "alice@example.com");
ps.executeUpdate();
ps.close();
conn.close();
```

23. Question: What is a ResultSet in JDBC and how is it used to retrieve data from a MySQL database? Briefly explain the use of next(), getString(), and getInt() methods with an example.

Answer: A ResultSet is an object that stores data retrieved from a SELECT query in JDBC.

Key Methods:

next(): Moves to the next row (return false if no more rows)

getString(column): Gets String value from a column.

getInt(column): Gets int value from a column.

Example:

```
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/mydb", "root", "password");
Statement stmt = conn.createStatement();
```



```

ResultSet rs = stmt.executeQuery("SELECT id, name
    FROM students");
while (rs.next()) {
    int id = rs.getInt("id");
    String name = rs.getString("name");
    System.out.println(id + ": " + name);
}
rs.close();
stmt.close();
conn.close();

```

24. Question: How does JPA manage the mapping between Java objects and relational tables?

Answer: JPA manages mapping between Java objects and relational tables using annotations. It abstracts SQL, making data access cleaner and object oriented.

Example:

```

import javax.persistence.*;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

```

```
private String name;
}
```

• @Entity → Marks class as a DB table

@Id → Marks primary key

@GeneratedValue → Auto generates ID

Advantages over JDBC :

JPA

Less boilerplate

Object-relational
mapping

Supports caching,
lazy load

DB-independent

JDBC

More code (SQL + mapping)

Manual result processing

Manual optimization needed

SQL needs changes per DB

25. Question: Describe the difference between the Entity Manager's `persist()`, `merge()`, and `remove()` operations

Answer: EntityManager Methods in JPA

Method	Purpose	When to use
<code>persist()</code>	Adds a new object to DB (insert)	When creating a new record.
<code>merge()</code>	Updates an existing object or saves detached one	When updating or reattaching.
<code>remove()</code>	Deletes an object from the DB	When deleting a record

Summary:

- Use `persist()` for new entries,
- `merge()` for updates or detached objects,
- `remove()` to delete records.

26. Question: Design a simple CRUD application using Spring Boot and MySQL to manage student records.

Answer:

Objective: Manage student records using Spring Data JPA Repository.

1. Entity:

@Entity

public class Student {

@Id

@GeneratedValue (strategy = GenerationType.IDENTITY)

private Long id;

private String name;

private String email;

}

2. Repository:

public interface StudentRepository extends JpaRepository < Student, Long > { }

3. CRUD operations in Controller

@RestController

@RequestMapping("/students")

public class StudentController {

 @Autowired

 private StudentRepository repo;

// create

@PostMapping

public Student create(@RequestBody Student s) {

 return repo.save(s);

}

// Read

@GetMapping("/{id}")

public Student read(@PathVariable Long id) {

 return repo.findById(id).orElse(null);

}

// Update

@PutMapping("/{id}")

public Student update(@PathVariable Long id,

 @RequestBody Student s) {

 s.setId(id);

 return repo.save(s);

}

```
// Delete
@DeleteMapping("/{id}")
public void delete(@PathVariable Long id) {
    repo.deleteById(id);
}
}
```

27. Question: How does Spring Boot simplify the development of RESTful services?

Answer: Spring Boot simplifies REST API developed by:

- Auto-configuring necessary components (Tomcat etc)
- Reducing boilerplate with annotations
- Easy JSON handling via @RequestBody and @ResponseBody
- Embedded server (no deployment hassle)

Example: REST Controller in Spring Boot

```
4. Student.java
public class Student {
    private String name;
    private int age;
}
```


2. StudentController.java

@RestController

@RequestMapping ("/students")

```
public class StudentController {
    private List <Student> list = new ArrayList <>();
```

@GetMapping

```
public List <Student> getAll () {
    return list;
}
```

@PostMapping

```
public String addStudent (@RequestBody student s) {
    list.add(s);
    return "Student added";
}
```

JSON Handling:

@RestController: Combines @Controller + @ResponseBody

@RequestBody: Binds JSON to Java object

@GetMapping, @PostMapping: Handle GET & POST

HTTP request

28. Question: Compare @RestController and @Controller in Spring Boot. In a RESTful API for a library system, describe how you would structure the endpoints for books using REST principles.

Answer:

Feature	@Controller	@RestController
Purpose	For MVC webapps	For REST APIs
Uses @ResponseBody	Must be added Manually	Automatically added to all methods
Return Type	Typically returns a view	Returns JSON/XML
Ideal Use Case	Web UI with Thymeleaf / JSP	RESTful services / APIs

RESTful API Design for a Library System - Books

Action	HTTP Method	Endpoint	Description
Get all books	GET	/books	Fetch list of all books
Get one book	GET	/books/{id}	Fetch details of a specific book

Add a book	POST	/books	Add a new book (JSON in body)
Update book	PUT	/books/{id}	Update existing book info
Delete book	DELETE	/books/{id}	Remove a book from library

29. Question: How does Maven manage dependencies and build lifecycle in a Spring Boot project? Explain the structure of a typical pom.xml and how the Spring Boot starter dependencies simplify development.

Answer: Maven manages:

Dependencies: Downloads and links required libraries automatically.

Build lifecycle: Defines phases like compile, test, package, deploy.

Typical pom.xml structure:

```
<project>
<modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId>
```

```
<artifactId> demo </artifactId>
<version> 1.0.0 </version>
```

```
<parent>
```

```
<groupId> org.springframework.boot </groupId>
```

```
<artifactId> spring-boot-starter-parent </artifactId>
```

```
<version> 3.0.0 </version>
```

```
</parent>
```

```
<dependencies>
```

```
<dependency>
```

```
<groupId> org.springframework.boot </groupId>
```

```
<artifactId> spring-boot-starter-web </artifactId>
```

```
</dependency>
```

```
<!-- ... Other dependencies ... -->
```

```
</dependencies>
```

```
<build>
```

```
<plugins>
```

```
<plugin>
```

```
<groupId> org.springframework.boot </groupId>
```

```
<artifactId> spring-boot-maven-plugin </artifactId>
```

```
</plugin>
```

```
</plugins>
```

```
</build>
```

```
</project>
```


How Spring Boot Starters Help :

- Starters are predefined dependency sets for common tasks. ~~star~~
- Simplifies setup by avoiding manual dependency management.
- Ensures compatible versions, reducing conflicts.

Question 30: Compare Maven and Gradle as build tools in Spring Boot projects. Discuss their syntax, performance, and dependency handling with an example of a build.gradle configuration for a simple REST API.

Answer: Maven vs Gradle in Spring Boot Projects

Aspect	Maven	Gradle
Syntax	XML-based (pom.xml)	Groovy / Kotlin DSL (build.gradle)
Performance	Slower, uses full builds by default.	Faster, supports incremental builds
Dependency management	Declarative, fixed versions.	More flexible, supports dynamic versions.
Build Customization	Limited and verbose	More powerful and concise scripting
Community & Tooling	Mature, wide IDE support	Growing fast, also well supported.

Example: build.gradle for Simple REST API

```
plugins {
```

```
    id 'org.springframework.boot' version '3.0.0'
```

```
    id 'io.spring.dependency-management' version '1.1.0'
```

```
    id 'java'
```

```
}
```

```
group = 'com.example'
```

```
version = '1.0.0'
```

```
sourceCompatibility = '17'
```

```
repositories {
```

```
    mavenCentral()
```

```
}
```

```
dependencies {
```

```
    implementation 'org.springframework.boot:spring-  
        -boot-starter-web'
```

```
    testImplementation 'org.springframework.boot:spring-  
        -boot-starter-test'
```

```
}
```